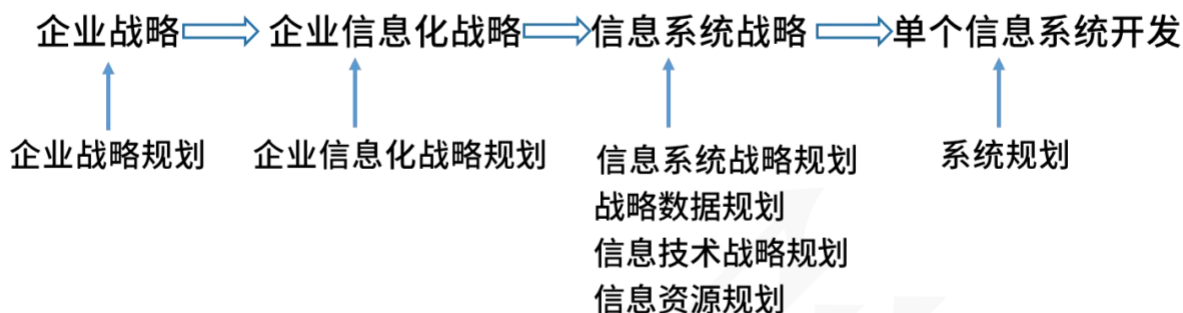


系统分析师易混淆知识点

易混淆点 1: 信息化战略体系规划



1、企业战略规划：企业战略规划是用机会和威胁评价现在和未来的环境，用优势和劣势评价企业现状，进而选择和确定企业的总体和长远目标，制定和抉择实现目标的行动方案。

希赛教育专家解读：确定企业未来发展的大方向。

2、信息系统战略规划：信息系统战略规划关注的是如何通过信息系统来支撑业务流程的运作，进而实现企业的关键业务目标，其重点在于对信息系统远景、组成架构、各部分逻辑关系进行规划。

希赛教育专家解读：为企业战略开发支撑系统。

3、信息技术战略规划：信息技术战略规划通常简称为 IT 战略规划，是在信息系统规划的基础上，对支撑信息系统运行的硬件、软件、支撑环境等进行具体的规划，它更关心技术层面的问题。

希赛教育专家解读：为支撑系统运行环境做规划。

4、信息资源规划：信息资源规划是在以上规划的基础上，为开展具体的信息化建设项目而进行的数据需求分析、信息资源标准建立、信息资源整合工作。

希赛教育专家解读：数据与标准相关的规划

5、系统规划：系统规划是信息系统生命周期的第一个阶段，其任务是对企业的环境、目标及现有系统的状况进行初步调查，根据企业目标和发展战略，确定信息系统的发展战略，对建设新系统的需求做出分析和预测，同时考虑建设新系统所受的各种约束，研究建设新系统的必要性和可能性。

希赛教育专家解读：单个项目的立项分析。

易混淆点 2: 磁盘阵列

RAID0（条块化）：性能最高，并行处理，无冗余，损坏无法恢复。

RAID1（镜像结构）：可用性、可修复性好，仅有 50%利用率。

RAID0+1（RAID0）：RAID0 与 RAID1 长处的结合，高效也可靠。

RAID3（奇偶校验并行传送）：N+1 模式，有固定的校验盘，坏一个盘可恢复。

RAID5（分布式奇偶校验的独立磁盘）：N+1 模式，无固定的校验盘，坏一个盘可恢复。RAID5 磁盘利用率为 $(n-1)/n$ ，具备容错功能。

RAID6（两种存储的奇偶校验）：N+2 模式，无固定的校验盘，坏两个盘可恢复。

注：RAID0 磁盘利用率为 100%，访问速度最快；RAID1 磁盘利用率为 50%，具备纠错功能；现在企业采用 RAID0 与 RAID1 结合的方式比较多。

易混淆点 3：企业应用集成

不同维度的集成划分：

(1) 按集成点划分：

	集成点	效果	解题关键点
界面集成	界面层	统一入口，产生“整体”感觉	“整体”感觉 最小代价实现一体化操作
数据集成	数据层	不同来源的数据逻辑或物理上“集中”	其他集成方法的基础
控制集成	应用逻辑层	调用其它系统已有方法，达到集成效果	/
业务流程集成 (过程集成)	应用逻辑层	跨企业，或优化流程而非直接调用	企业之间的信息共享能力
门户集成	/	将内部系统对接到互联网上	发布到互联网上

(2) 按传输方式划分：

	特点
消息集成	数据量小，交互频繁，立即地，异步
共享数据库	交互频繁，立即地，同步
文件传输	数据量大，交互频度小，即时性要求低（月末，年末）

易混淆点 4：电子政务

电子政务主要有 3 类角色：政府（Government）、企（事）业单位（Business）及公民（Citizen）。如果有第 4 类就是公务员（Employee）。

类型	应用
G2G	基础信息的采集、处理和利用，如：人口信息、地理信息 各级政府决策支持

G2E	政府内部管理系统
G2B	政府给企业单位颁发【各种营业执照、许可证、合格证、质量认证】
B2G	企业向政府缴税 企业向政府提供各种商品和服务【含竞/投标】 企业向政府提建议，申诉
G2C	社区公安和水、火、天灾等与公共安全有关的信息 户口、各种证件和牌照的管理
C2G	个人应向政府缴纳的各种税款和费用 个人向政府反馈民意【征求群众意见】 报警服务【盗贼、医疗、急救、火警等】

易混淆点 5：总线的分类

分类依据	类型
功能划分	数据总线、地址总线和控制总线
位置划分	机内总线（芯片内总线、系统总线）、机外总线
功用划分	局部总线（ISA 总线和 CPU 之间）、系统总线（系统内部各插件信息线）、通信总线（远程、测试连接等）
总线数据线数量	并行总线和串行总线

易混淆点 6：投资回收期

1、静态投资回收期（不考虑货币的时间价值因素）

如果投资在建设期 m 年内分期投入， t 年的投资为 P_t ， t 年的净现金收入为 $(CI - CO)_t$ ，则能够使下面公式成立的 T 即为静态投资回收期。

$$\sum_{t=0}^m P_t = \sum_{t=0}^T (CI - CO)_t$$

2、动态投资回收期（考虑资金的时间价值因素）

如果考虑资金的时间价值，则动态投资回收期 I 的计算公式，应满足

$$\sum_{t=0}^{T_p} \frac{(CI - CO)_t}{(1 + i)^t} = 0$$

计算动态投资回收期的实用公式为: $T_p = \text{累计折现值开始出现正值的年份数} - 1 + |\text{上年累计折现值}| / \text{当年折现值}$

- 3、投资回收率=1/投资回收期*100%
- 4、投资回报率=投资收益率=投资收益/投资总额*100%
- 5、年均投资收益率=运营期年均净收益/投资总额*100%
- 6、现值指数=投资净收益/投资成本*100%=净现值率+100%

注:做题时需要仔细审题,题目要求的是动态投资回收期还是静态投资回收期

易混淆点 7: 软件开发模型

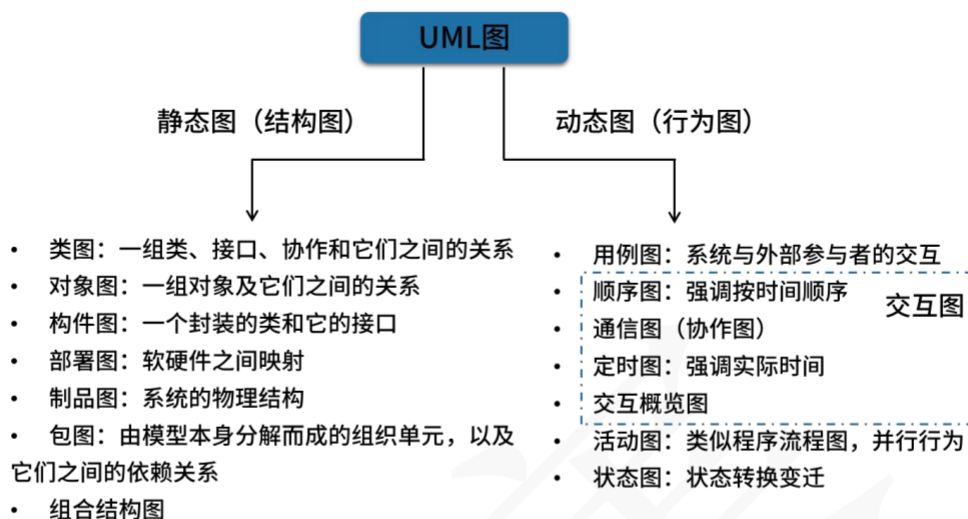
模型	描述	特点
【瀑布模型】 以软件需求完全 确定为前提	一种严格定义方法,它将软件开发的过程分为软件计划、需求分析、软件设计、程序编码、软件测试和运行维护 6 个阶段(严格区分阶段),形如瀑布流水,最终得到软件产品。	依赖于需求不能适应变化;单一流程,开发中的经验不能反馈应用于本过程;风险迟至后期开发阶段才显露,不能及早纠正,对于项目风险的控制能力较弱。项目常常延期完成,开发费用超出预算。
【V 模型】 瀑布的变种	V 模型从左到右,描述了基本的开发过程和测试行为,其价值在于它明确地标明了测试过程中存在的不同级别,并且清楚地描述了这些测试阶段和开发过程各阶段的对应关系。	强调软件开发的协作和速度,将软件实现和验证有机地结合起来,在保证较高的软件质量情况下缩短开发周期。适合企业级的软件开发。
【螺旋模型】 原型+瀑布 强调风险	一种演化软件过程模型,它将原型实现的迭代特征与线性顺序模型中控制的和系统化的方面结合起来,使软件的增量版本的快速开发成为可能。每次迭代都包括制订计划、风险分析、实施工程和客户评估四个方面的工作。	支持用户需求的动态变化,提高了软件的适应能力,为用户关键决策提供方便,为项目管理人员及时调整管理决策提供便利,降低软件开发风险。需要开发人员具有风险评估经验和专门知识。过多迭代次数会增加开发成本,延迟提交时间。特别适用于庞大、复杂并具有高风险的系统。
【喷泉模型】 早期面向对象模 型	喷泉模型是一种以用户需求为动力,以对象为驱动力的模型,主要用于描述面向对象的软件开发过程。	迭代,无间隙。

【演化模型】 需求不完整	在快速开发一个原型的基础上,根据用户在调用原型的过程中提出的反馈意见和建议,对原型进行改进,获得原型的新版本,重复这一过程,直到演化成最终的软件产品。	任何功能一经开发就能进入测试,以便验证是否符合产品需求,可以帮助引导出高质量的产品要求。但如果不加控制地让用户接触开发中尚未稳定的功能,可能对开发人员及用户都会产生负面的影响。
【变换模型】 基于形式化规格说明语言和程序变换	以形式化规格说明为基础开发一个软件原型,用户可以从人机界面、系统主要功能和性能等方面对原型进行评审。必要时,可以修改软件需求、形式化规格说明和原型,直至原型被确认为止。	优点是解决了代码结构经多次修改而变坏的问题,减少了许多中间步骤。但是,变换模型仍有较大局限,以形式化开发方法为基础的变换模型需要严格的数学理论和一整套开发环境的支持。
【智能模型】 基于知识	综合了上述若干模型,并把专家系统结合在一起。该模型应用基于规则的系统,采用规约和推理机制,帮助开发人员完成开发工作,并使维护在系统规格说明一级进行。需要建立知识库,将模型本身、软件工程知识与特定领域的知识分别存入知识库。	

易混淆点 8: 需求获取方法

方法	特点
用户访谈	1 对 1-3, 有代表性的用户, 了解主观想法, 交互好。成本高, 要有领域知识支撑
问卷调查	用户多, 无法一一访谈, 成本低
采样 (抽样调查)	基于数理统计, 降低成本, 快速获取 采样技术包括: 随机采样、分层采样、聚类采样等, 具体采用哪种方法根据文档数量、样本大小和实际情况决定
情节串联板 【原型前身】	一系列图片, 通过这些图片来讲故事。类型包括: 被动式、主动式和交互式
联合需求计划	高度组织的群体会议, 各方参与, 了解想法, 消除分歧, 交互好, 成本高
需求记录技术	需求记录工具有: 任务卡片、场景说明、用户故事和 Volere 白卡等

易混淆点 9: UML 图



UML 2.0 包括 14 种图，分别如下：

(1) 类图（class diagram）。类图描述一组类、接口、协作和它们之间的关系。类图给出了系统的静态设计视图，活动类的类图给出了系统的静态进程视图。

(2) 对象图（object diagram）。对象图描述一组对象及它们之间的关系。对象图描述了在类图中所建立的事物实例的静态快照。对象图从真实案例或原型案例的角度建立的。

(3) 构件图（component diagram）。构件图描述一个封装的类和它的接口、端口，以及由内嵌的构件和连接件构成的内部结构。构件图用于表示系统的静态设计实现视图。

(4) 组合结构图（composite structure diagram）。组合结构图描述结构化类（例如，构件或类）的内部结构，包括结构化类与系统其余部分的交互点。组合结构图用于画出结构化类的内部内容。

(5) 用例图（use case diagram）。用例图描述一组用例、参与者及它们之间的关系。用例图给出系统的静态用例视图。

(6) 顺序图（sequence diagram）。顺序图是一种交互图（interaction diagram），交互图展现了一种交互，它由一组对象或参与者以及它们之间可能发送的消息构成。交互图专注于系统的动态视图。顺序图是强调消息的时间次序的交互图。

(7) 通信图（communication diagram）。通信图也是一种交互图，它强调收发消息的对象或参与者的结构组织。

(8) 定时图（timing diagram）。定时图也是一种交互图，它强调消息跨越不同对象或参与者的实际时间，而不仅仅是关心消息的相对顺序。

(9) 状态图 (state diagram)。状态图描述一个状态机, 它由状态、转移、事件和活动组成。状态图给出了对象的动态视图。它对于接口、类或协作的行为建模尤为重要, 而且它强调事件导致的对象行为, 这非常有助于对反应式系统建模。

(10) 活动图 (activity diagram)。活动图将进程或其他计算结构展示为计算内部一步步的控制流和数据流。活动图专注于系统的动态视图。它对系统的功能建模和业务流程建模特别重要, 并强调对象间的控制流程。

(11) 部署图 (deployment diagram)。部署图描述对运行时的处理节点及在其中生存的构件的配置。部署图给出了架构的静态部署视图, 通常一个节点包含一个或多个部署图。

(12) 制品图 (artifact diagram)。制品图描述计算机中一个系统的物理结构。制品包括文件、数据库和类似的物理比特集合。制品图通常与部署图一起使用。制品也给出了它们实现的类和构件。

(13) 包图 (package diagram)。包图描述由模型本身分解而成的组织单元, 以及它们之间的依赖关系。

(14) 交互概览图 (interaction overview diagram)。交互概览图是活动图和顺序图的混合物。

希赛专家解读:

1、活动图与流程图关系

活动图描述的是对象活动的顺序关系所遵循的规则, 它着重表现系统的行为, 而非处理过程; 而流程图着重描述处理过程。

流程图一般都限于顺序进程, 而活动图则可以支持并发进程。

活动图是面向对象的, 而流程图是面向过程的。

2、活动图与状态图对比

状态图主要用于描述一个对象在其生存期间的动态行为, 表现一个对象所经历的状态序列, 引起状态转移的事件 (event), 以及因状态转移而伴随的动作 (action)。

活动图可以用于描述系统的工作流程和并发行为。活动图其实可看作状态图的特殊形式, 活动图中的一个活动结束后将立即进入下一个活动 (在状态图中状态的转移可能需要事件的触发)。

两者最大的区别是: 状态图侧重于描述行为的结果, 而活动图侧重描述行为的动作。其次活动图可描述并发行为, 而状态图不能。

易混淆点 10: UML 图关系

(1) 用例关系包括: 包含关系、扩展关系、泛化关系。

1、包含关系：其中这个提取出来的公共用例称为抽象用例，而把原始用例称为基本用例或基础用例系：当可以从两个或两个以上的用例中提取公共行为时，应该使用包含关系来表示它们。

2、扩展关系：如果一个用例明显地混合了两种或两种以上的不同场景，即根据情况可能发生多种分支，则可以将这个用例分为一个基本用例和一个或多个扩展用例，这样使描述可能更加清晰。

3、泛化关系：当多个用例共同拥有一种类似的结构和行为的时候，可以将它们的共性抽象成为父用例，其他的用例作为泛化关系中的子用例。在用例的泛化关系中，子用例是父用例的一种特殊形式，子用例继承了父用例所有的结构、行为和关系。

(2) 类图/对象图关系：

1、依赖关系：一个事物发生变化影响另一个事物。

2、泛化关系：特殊/一般关系

3、关联关系：描述了一组链，链是对象之间的连接。

4、聚合关系：整体与部分生命周期不同。

5、组合关系：整体与部分生命周期相同。

6、实现关系：接口与类之间的关系

注：用例关系只有包含、扩展和泛化，不要和类图的关系弄混淆。

易混淆点 11：设计模式

(1) 创建型模式

设计模式名称	简要说明	速记关键字
Factory Method 工厂方法模式	定义了创建对象的接口，它允许子类决定实例化哪个类	动态生产对象
Abstract Factory 抽象工厂模式	提供一个接口，可以创建一系列相关或相互依赖的对象，而无需指定它们具体的类	生产成系列对象
Builder 构建器模式	将一个复杂类的表示与其构造相分离，使得相同的构建过程能够得出不同的表示	复杂对象构造
Prototype 原型模式	允许对象在不了解要创建对象的确切类以及如何创建等细节的情况下创建自定义对象。通过拷贝原型对象来创建新的对象	克隆对象
Singleton 单例模式	确保一个类只有一个实例，并且提供了对该类的全局访问入口	单实例

(2) 结构型模式

设计模式名称	简要说明	速记关键字
Adapter 适配器模式	将一个类的接口转换成用户希望得到的另一种接口。它使原本不相容的接口得以协同工作	转换接口
Bridge 桥接模式	将一个复杂的组件分成两个独立的但又相关的继承层次结构 将类的抽象部分和它的实现部分分离开来, 使它们可以独立地变化	继承树拆分
Composite 组合模式	创建树型层次结构来改变复杂性, 同时允许结构中的每一个元素操作同一个接口。用于表示“整体-部分”的层次结构	树形目录结构
Decorator 装饰模式	在不修改对象外观和功能的情况下添加或者删除对象功能 即动态地给一个对象添加一些额外的职责	动态附加职责
Facade 外观模式	子系统中的一组接口提供了一个统一的接口	对外统一接口
Flyweight 享元模式	可以通过共享对象减少系统中低等级的、详细的对象数目 提供支持大量细粒度对象共享的有效方法	汉字编码
Proxy 代理模式	为控制对初始对象的访问提供了一个代理或者占位符对象	快捷方式

(3) 行为型模式

设计模式名称	简要说明	速记关键字
Chain of Responsibility 职责链模式	可以在系统中建立一个链, 这样消息可以在首先接收到它的级别处被处理, 或者可以定位到可以处理它的对象	传递职责
Command 命令模式	在对象中封装了请求, 这样就可以保存命令, 将该命令传递给方法以及像任何其他对象一样返回该命令	日志记录, 可撤销
Interpreter 解释器模式	可以解释定义其语法表示的语言, 还提供了用表示来解释语言中的语句的解释器	虚拟机的机制
Iterator 迭代器模式	为集合中的有序访问提供了一致的方法, 而该集合是独立于基础集合, 并与之相分离的	数据集
Mediator 中介者模式	通过引入一个能够管理对象间消息分布的对象, 简化了系统中对象间的通信	不直接引用
Memento	保持对象状态的“快照”(snapshot), 这样对象可以在	游戏存档

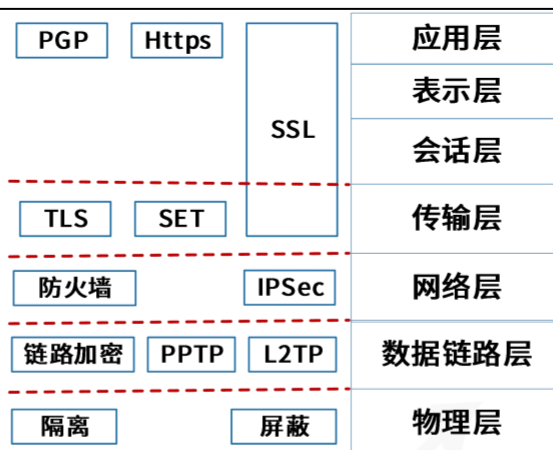
备忘录模式	不向外界公开其内容的情况下返回到它的最初状态	
Observer 观察者模式	为组件向相关接收方广播消息提供了灵活的方法 定义对象间的一种一对多的依赖关系	订阅、广播、联动
State 状态模式	允许一个对象在其内部状态改变时改变它的行为	状态变成类
Strategy 策略模式	定义一系列算法，把它们一个个封装起来，并且使它们之间可互相替换，从而让算法可以独立于使用它的用户而变化	多方案切换
Template Method 模板方法模式	提供了在不重写方法的前提下允许子类重载部分方法的方法	框架
Visitor 访问者模式	提供了一种方便的、可维护的方法来表示在对象结构元素上要进行的操作	数据与操作分离

易混淆点 12: 软件维护

- 1、正确性维护【修 BUG】：识别和纠正软件错误/缺陷，测试不可能发现所有错误。
- 2、适应性维护【应变】：指使应用软件适应环境变化【外部环境、数据环境】而进行的修改。
- 3、完善性维护【新需求】：扩充功能和改善性能而进行的修改。
- 4、预防性维护【针对未来】：为了适应未来的软硬件环境的变化，应主动增加预防性的新的功能，以使用系统适应各类变化而不被淘汰。经典实例：【专用】改【通用】。

注：适应性维护和预防性维护比较像，要注意区分。一个是适应环境变化，一个是适应未来的环境变化

易混淆点 13: 安全协议



PGP (Pretty Good Privacy) : 针对邮件和文件的混合加密系统。

SSL (Secure Sockets Layer) : 工作在传输层至应用层。

TLS (Transport Layer Security) : 传输层安全协议。

SET (Secure Electronic Transaction) : 安全电子交易协议。电子商务, 身份认证。普遍的说法是将其归为应用层。

IPsec (Internet Protocol Security) : 对 IP 包加密。

注: SSL 协议工作于应用层至传输层, 但却属于传输层协议哦。

易混淆点 14: 系统测试

系统测试包括: 功能测试、性能测试、健壮性测试、用户界面测试、安全性测试、安装与反安装测试等。

其中性能测试又可以进一步分类为:

负载测试: 各种工作负载下系统的性能

压力测试【测上限】: 系统的瓶颈或不能接受的性能点

强度测试【测下限】: 系统资源特别低的情况下运行

容量测试【并发测试】: 同时在线的最大用户数

可靠性测试: MTTF 之类的参数。

注: 主要区分压力测试的强度测试, 一个是测上限, 一个测下限, 很容易混淆

易混淆点 15: 嵌入式实时操作系统调度算法

(1) 基于优先级的抢占调度: 基于优先级的抢占调度在任何时候运行的任务都是所有就绪任务中具有最高优先级的任务。优先级可以分为静态优先级和动态优先级。

(2) 时间轮转调度算法：为每个任务提供确定份额的 CPU 执行时间。

(3) 单调速率调度 (Rate Monotonic Scheduling, RMS) 算法：静态固定优先级算法，任务周期越短优先级越高，是静态调度中最优的。

(4) 截止时间优先 (Earliest Deadline First, EDF) 调度算法 (最早截止期调度算法)：根据任务的截止时间来确定其优先级，对于时间期限最近的任务，分配最高的优先级。在动态调度算法中 CPU 的利用率达到了最高。

(5) 最低松弛度优先 (Least Laxity First, LLF) 算法：根据任务紧急 (或松弛) 的程度，来确定任务的优先级。任务的紧急程度愈高，为该任务所赋予的优先级就愈高，使之优先执行。动态调度算法。

(6) 基于分区 (partition) 化的分区 APPS 调度算法：将处理器时间资源划分成多个独立时间片资源，以供多个分区实体独立享用。通过遵循特定的分区调度原则和时间优先调度 (TPS) 策略，该算法可以确保系统的稳定性和性能。静态调度算法。

易混淆点 16：负载均衡动态算法和静态算法

(1) 静态算法 (不考虑动态负载)：

- 1、轮转算法：轮流将服务请求 (任务) 调度给不同的节点 (即：服务器)。
- 2、加权轮转算法：考虑不同节点处理能力的差异。
- 3、源地地址哈希散列算法：根据请求的源 IP 地址，作为散列键从静态分配的散列表找出对应的节点。
- 4、目标地址哈希散列算法：根据请求目标 IP 做散列找出对应节点。
- 5、随机算法：随机分配，简单，但不可控。

(2) 动态算法 (考虑动态负载)

- 1、最小连接数算法：每个节点处理能力相同的情况下，新请求分配给当前活动请求数量最少的节点。
- 2、加权最小连接数算法：考虑节点处理能力不同，按最小连接数分配。
- 3、加权百分比算法：考虑了节点的利用率、硬盘速率、进程个数等，使用利用率来表现剩余处理能力。

制作于 25 年 7 月 适用于第 2 版教材